

Beyond Before-and-After:

Process-Quality Evaluation of Refactoring Sessions

Scoring the path a developer takes through a refactoring session, and generating reference trajectories where a better path existed.

Ethan Hosier · Imperial College London · Department of Computing

Supervisor: Dr Robert Chatley · Second marker: Dr Cristian Cadar

Same destination, different journeys

*Two developers refactor the same code **and arrive at the same final state.***

Developer A

- Ran tests.
- Committed checkpoints.
- Used IDE refactorings.

Developer B

- Broke the build twice.
- Added then removed 80 lines.
- Hand-edited what the IDE could do safely.

- Which one would you want on your team?
- Endpoint-only evaluation cannot tell them apart.

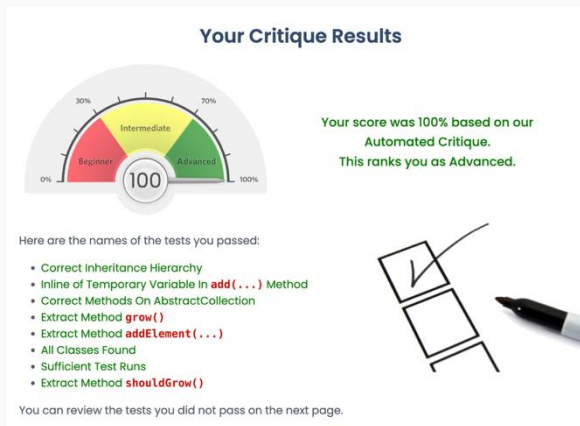
Motivation: what's missing in prior work

To help developers refactor better, we need to measure what a good refactoring session actually looks like - not just what code it produces.

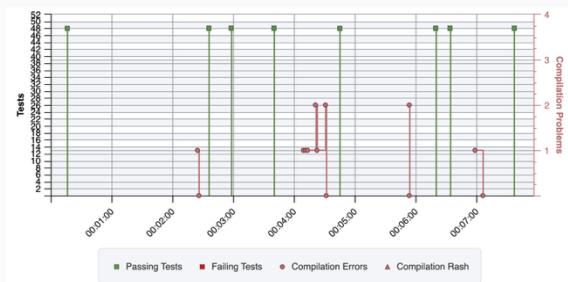
But prior work mostly measures outcomes, not the process:

- **Endpoint quality metrics** (*CK, smells, cohesion, coupling*) - compare before vs after; ignore the path between, and can't distinguish two sessions ending at the same code.
- **Refactoring recommenders / sequence generators** (*search-based, ReSynth*) - generate paths toward a target end state; don't score the developer's own path.
- **Exercise-scoped feedback tools** - match explicit events inside predefined exercises; don't generalise to arbitrary sessions.

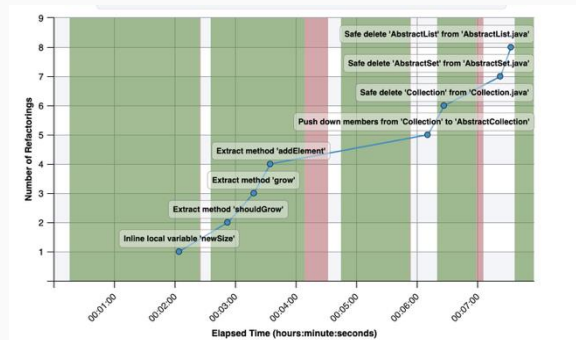
Closest Existing Tool... Industrial Logic



Score



Compilation + tests



Refactorings

Gap: no system scores an arbitrary observed refactoring session against an explicit process-quality metric, and shows where a better path was available.

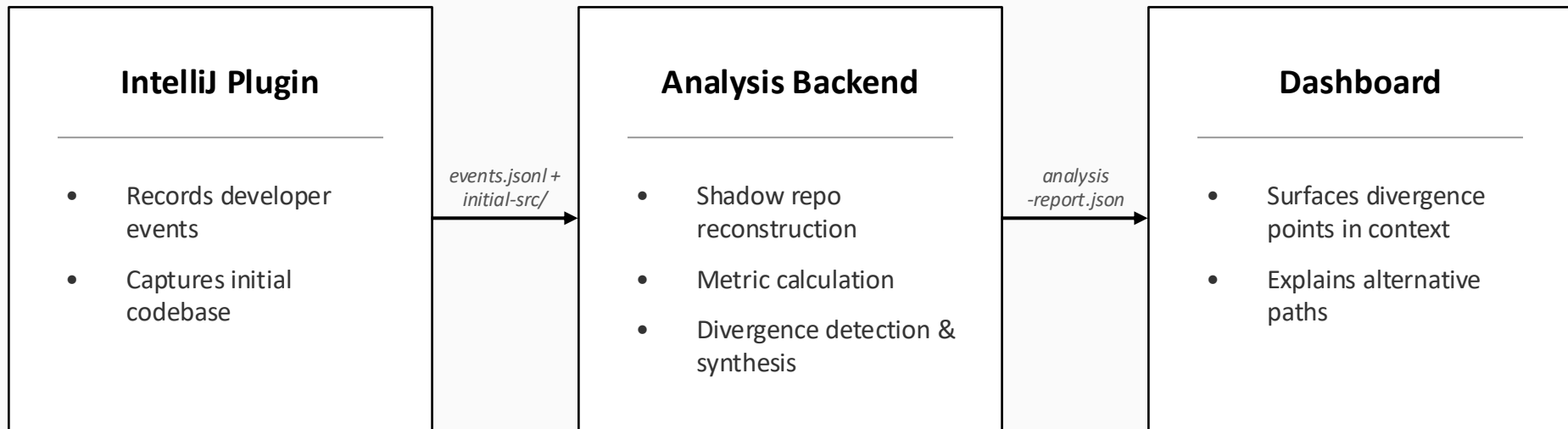
Narrowing the design space

Evaluating refactoring as a process is a wide, mostly-unexplored design space. Four requirements shaped what we built.

- **Quantifiable + comparable score** - one number per trajectory so sessions can be compared. [1]
- **Actionable, moment-specific feedback** - point to a specific moment AND a "what you could have done instead" alternative. [2]
- **Efficient alternative synthesis** - the state-action space over a real codebase is too large to enumerate, so we must find efficient methods of identifying "better" alternatives.
- **Seamless, in-workflow capture** - zero extra developer effort to use the tool. [3]

[1] Composite quality models (Quamoco, QMOOD), adapted from snapshot- to trajectory-level. [2] Inverts prior refactoring recommenders (ReSynth, Refactoring Navigator, search-based refactoring). [3] IDE-event logging (Damevski et al., CodeWatcher).

The tool, end to end: actionable refactoring process analysis



Demo

OrderProcessor: Breaking up a long method into smaller methods

Scoring a trajectory: $J(\tau)$

$$J(\tau) = \text{clip}_{[0,100]} \left(50 \text{ (baseline)} + \right. \\ \begin{aligned} &+50 \times \text{cleanliness gain}^* \\ &+11 \times \text{step-savings bonus} \\ &-28 \times \text{broken build / test rate} \\ &-14 \times \text{skipped-test rate} \\ &-11 \times \text{manual edits the IDE could refactor} \\ &-11 \times \text{Intermediate cleanliness lag (degradation)} \\ &-7 \times \text{Long durations without commits} \end{aligned} \left. \right)$$

** cleanliness gain = mean of 6 normalised signals: complexity, coupling, duplication, readability, smells, cohesion.
All 6 weighted equally - no trajectory-level calibration data exists, so equal weights are the least-assumptive choice.*

Four kinds of divergence

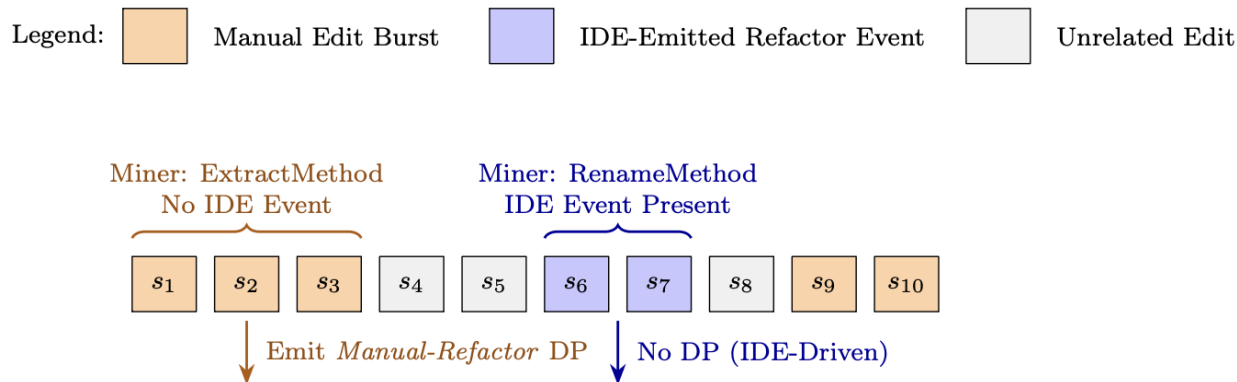
- Manual-Refactor - unnecessary risk (hand-edit when the IDE could have safely done it).
- Ordering - right refactorings, wrong sequence.
- Rework - unnecessary churn (added then removed).
- Hygiene - missing safety checkpoints (no tests, no commits).

Manual-Refactor: detect + synthesise

“Very actionable - at the start I didn't know most IntelliJ actions.”

— P2

- Developer hand-edited something the IDE could have done safely (and with its precondition checks).
- Refactoring Miner on sliding commit windows (of shadow repo) - anything we find that the IDE did not emit an event for is a manual refactoring.
 - Apply the equivalent IDE refactoring (Eclipse / JDT).
 - Reconcile minor JDT $\leftrightarrow$ IntelliJ AST differences (e.g. static modifiers, variable liveness).
 - Merge the user's other edits back on top.



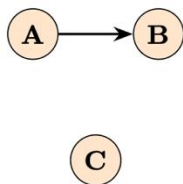
Ordering: detect + synthesise

"How can you actually apply that in the future without knowing beforehand?"

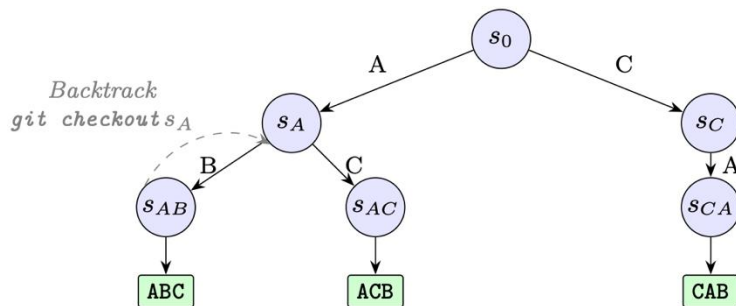
— P2

- Developer performed their refactoring steps in a sub-optimal order, leading to unnecessary intermediate degradation.
1. Identify dependencies between steps → DAG.
 2. Enumerate valid topological orderings.
 3. Synthesise (Eclipse / JDT), utilising Prefix Trie (DFS) and git checkouts on backtrack.
 4. Normalize AST + compare with original.

Dependency DAG



Prefix Trie (DFS)

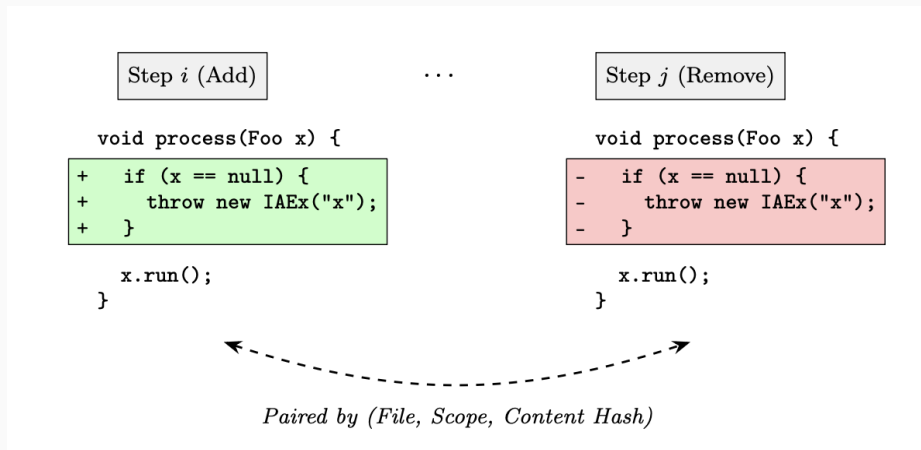


Rework: detect + synthesise

"Over-penalised undos + redos."

— P1

- Code added and later removed (or vice versa) - unnecessary churn.
 1. Compare git diffs between adjacent steps of shadow repo.
 2. Hash each hunk, and look for a matching, opposite hunk with matching (file, scope, content-hash) in a future git diff.
 3. Remove that hunk from both diffs, calculate the updated line numbers of every git diff hunk between each of the intermediate states, and replay these new diffs from the start step.



Hygiene: detect + synthesise

“The most actionable kind - committing more frequently in particular.”

— P1

- Long stretches of work without test runs, or commits.
 1. Iterate through each run's IDE events.
 2. If there has been > 1 minute since the last edit and still no test run, flag a Test DP.
 3. If there has been > 5 refactoring steps since the last commit, flag a Commit DP.
 4. For each DP, generate an identical run but with the corresponding test or commit event.
- Very cheap to compute.

What we wanted to evaluate

- **3 Questions**
 - Is the tool accurate?
 - Is the process score reliable?
 - Does the tool change developer behaviour?
- **2 Datasets**
 - 45-session labelled injection set - controlled, balanced per-kind coverage.
 - 30-session randomised user study - 5 MEng participants × 6 sessions, 2 arms (feedback vs no-feedback).

***Reproducible:** every table and figure is regenerated from the Jupyter notebook in the repo.*

Experiment 1: Is the tool accurate?

Experiment 1: Is the tool accurate?

DETECTION

1.00

Precision · all four kinds

When the detector flags a divergence, it's never a false positive. (45 labelled sessions, three raters at Cohen's $\kappa \geq 0.72$.)

1.00 / 1.00 / 0.76 / 0.36

Recall · Hygiene / Rework / MR / Ordering

Three kinds catch everything; Ordering's validator is deliberately conservative - a known scope limit.

SYNTHESIS

62%

Alternatives beat user trajectory · 41 of 66

When the tool flags a divergence, the synthesised alternative usually scores higher than the path the user actually took.

100% / 75% / 46% / 42%

Beat rate by kind · Hygiene / MR / Rework / Ordering

Hygiene and Manual-Refactor reliably improve the score; Ordering ties often because alternatives reach the same end state.

Precision-perfect detection, balanced recall (one known scope limit on Ordering), and a higher-scoring alternative in roughly two-thirds of detected moments.

Experiment 2: Is the process score reliable?

Experiment 2: Is the process score reliable?

STABILITY

96.5%

Single-knob top-1 preserved

Top-ranked divergence rarely changes under any one-weight perturbation. Only 1.8% of cases are clamp-frozen, so this is genuine response - not a clamp artefact.

ABLATION

7 / 7

**Process terms with measurable rank effect · LOO τ -b range
0.527 - 0.784**

Length is the strongest rank-carrier (0.527); lag the weakest (0.784, but contributes via magnitude). No process term is redundant.

84.6%

Multi-knob top-1 preserved · mean τ -b = 0.586

When all 7 weights move at once ($\sigma = \ln 2 \approx \times 0.25$ to $\times 4$), the top-ranked divergence holds in ~85% of samples. Meaningful rank correlation, not full preservation.

$\leq 0.9\%$

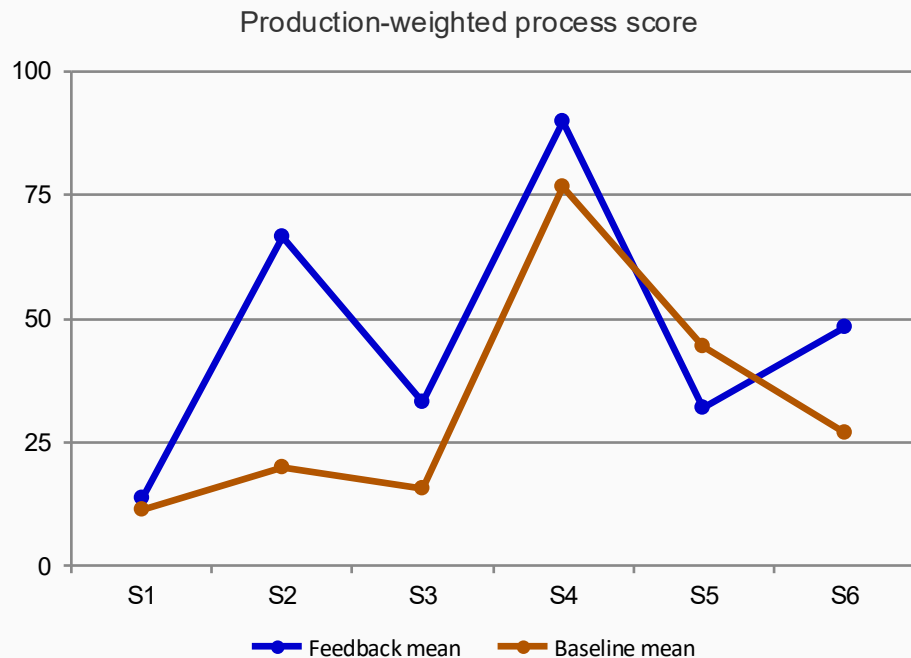
Top-1 disruption from any single cleanliness sub-weight

Perturbing any of the 6 cleanliness signals (complexity, coupling, duplication, readability, smells, cohesion) barely shifts the ranking - ~10× less than process weights. Equal weights are defensible.

Top-1 holds under both single- and multi-knob perturbation, and every process term carries measurable rank effect. Robustness $\neq$ calibration: weights aren't validated against downstream outcomes.

Experiment 3: Does feedback change developer behaviour?

Fewer divergences with feedback - but the raw score is noisy



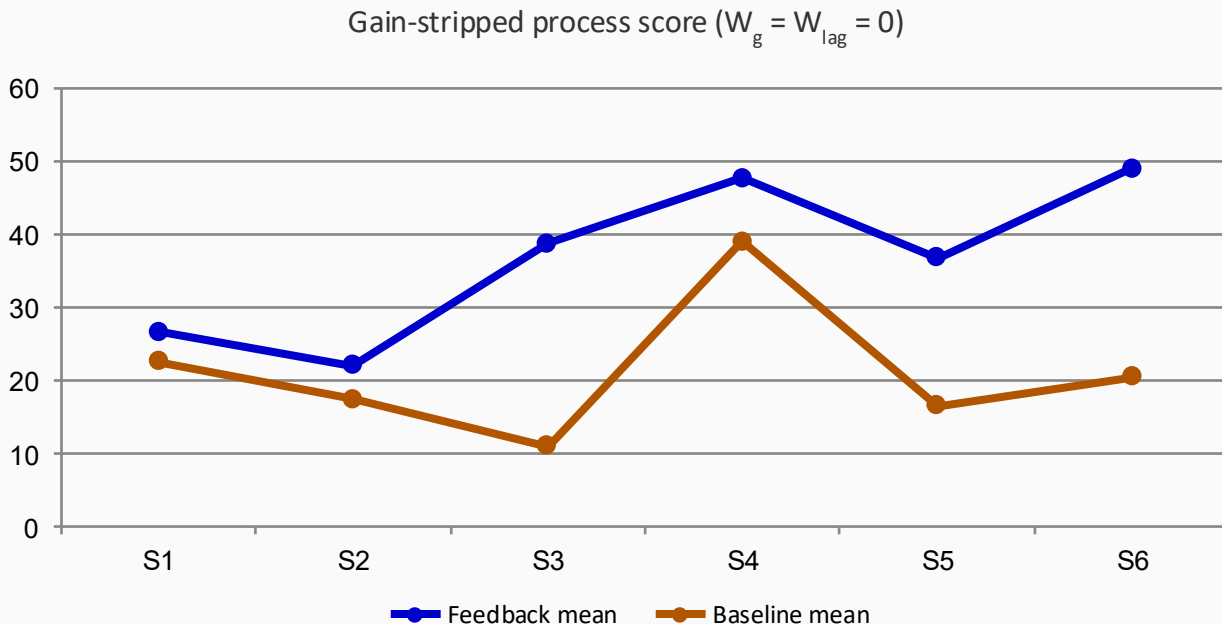
2.7 vs 6.0

DPs per session per participant · feedback vs baseline

Participants who saw feedback flagged a divergence less than half as often. 2.2× difference across the 30-session study.

But the production-weighted score is dominated by task difficulty - both groups peak at S4. The behavioural signal needs the gain-stripped view (next slide).

Strip the task signal - process discipline emerges

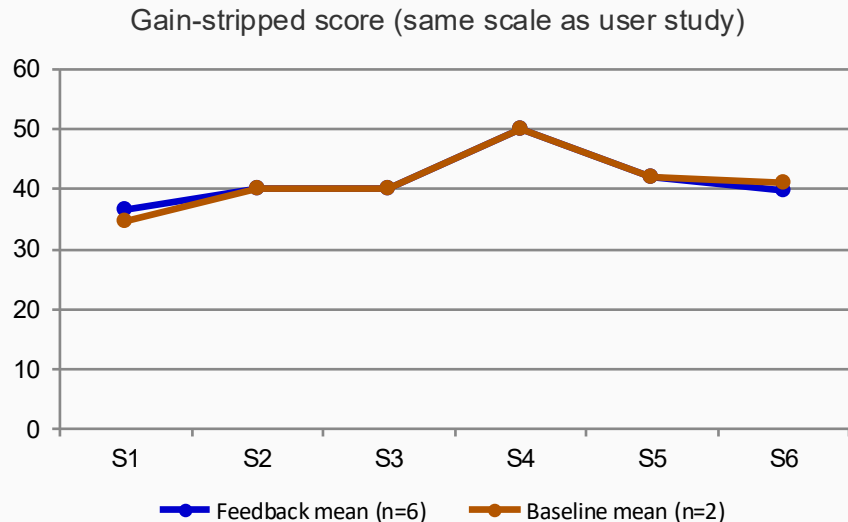


Once cleanliness gain is removed, the score reflects *how disciplined the process was* - tests run, IDE refactorings used, commits at sensible points. The feedback group climbs **+22.3** across the arc; the baseline group moves **-2.0**.

n = 3 vs n = 2 - directional finding, not a hypothesis test.

Extension: tool doesn't transfer cleanly to agent traces

48 sessions, 8 agents (Claude / GPT / Gemini × Claude Code / OpenCode / Cursor CLI), same playbook as the user study.



Kind	DPs	Why under-fires
Manual-Refactor	41	text edits, not IDE menu
Hygiene	0	batched commits don't trip threshold
Rework	0	edit bursts collapse to one
Ordering	1	same burst-collapse

Score is blind to the difference: feedback ΔJ +3.0 vs baseline +6.5. Yet 5 of 6 agents wrote explicit "I'll do X next session" plans citing prior warnings - the detectors just can't see those changes happen.

Scope limit by design - adapting detectors for agent traces is future work.

Conclusion

Process-aware refactoring evaluation is tractable, and a small randomised study gives directional evidence that feedback improves measured process discipline.

- A process-quality metric on *arbitrary sessions*
- A divergence-point detector & synthesiser of alternative trajectories.
- Three datasets - 45 injection sessions, 30-session user study, 48-session agent extension.
- A working end-to-end research prototype: IntelliJ plugin, analysis backend, and dashboard.

Not yet established: externally calibrated measure of refactoring quality, statistically conclusive behavioural effect, or generalisation beyond Java / IntelliJ / human traces.

Q & A